

Mohaned Bouzaïdi, Loïc Dreux, Frédéric Vézina

Algorithmes Vidéoludiques

PGJ1302, Groupe 65

Rapport Technique du travail pratique

Travail présenté à Simon Dansereau

Université du Québec en Abitibi-Témiscamingue

10 décembre 2025

Contents

1. Structures de données utilisées	3
1.1 La File d'actions (std::deque)	3
1.2 Structure d'Action (struct Action).....	3
1.3 L'inventaire (std::deque<Item>).....	3
1.4 Structure d'Items (struct Item)	4
2. Choix techniques et Architecture.....	4
2.1 Intégration de SFML 3.0	4
2.2 Boucle de Jeu et DeltaTime (dt).....	4
2.3 Machine à État Simple (isBusy).....	4
2.4 Parsing de Commandes (std::stringstream).....	5
3. Problèmes rencontrés	5
3.1 Gestion des DLLs et Git	5
3.2 Gestion des Items dans l'inventaire	5
3.3 problème loïc.....	Erreur! Signet non défini.
4. Répartition des tâches	6
[Mohaned] : Architecture Moteur & Rendu	6
[Loïc] : Ennemis, npc, combat, dialogue et game over	6
[Frédéric] : Items, Inventaires, get et use, items à la mort d'un ennemi.....	7

1. Structures de données utilisées

1.1 La File d'actions (std::deque)

Pour gérer la liste des commandes en attente, nous devons utiliser une structure de type "Queue" (FIFO - First In, First Out). Bien que la librairie standard offre std::queue, j'ai opté pour l'utilisation d'une **std::deque (Double-ended queue)** comme conteneur sous-jacent.

Justification :

Le devis exigeait un "Affichage de la file d'actions" (HUD) en temps réel sur l'écran. Une std::queue standard ne permet pas d'itérer sur ses éléments (on ne peut voir que le premier et le dernier). En utilisant une std::deque, nous conservons la rapidité d'insertion et de retrait ($O(1)$ aux extrémités), mais nous gagnons la capacité de parcourir la liste complète pour l'afficher graphiquement à chaque frame via la méthode getAll().

1.2 Structure d'Action (struct Action)

Plutôt que d'utiliser de l'héritage complexe pour chaque type d'action, j'ai implémenté une structure légère Action contenant :

- Un enum class ActionType pour identifier rapidement le type (Move, Wait, Attack, etc.).
- Des données génériques (un sf::Vector2f pour la position, un float pour la durée).
- Une std::string description préformaté lors de la création pour faciliter l'affichage dans l'interface.

1.3 L'inventaire (std::deque<Item>)

Pour gérer les items dont le joueur possède, j'ai créé une std::deque contenant des items. Pour faire l'inventaire, j'ai simplement réutilisé une partie du code de la file d'actions, étant donné qu'un inventaire est comme la file d'actions, mais bien entendu, avec le fait que l'état des items change seulement quand un joueur en ramasse un nouveau ou en utilise un. L'inventaire est donc, tout comme la file d'actions, une queue de type First In First Out.

Justification:

J'avais simplement besoin d'une queue d'items qui pouvait rapidement être mise à jour pour soit ajouter ou supprimer des items. Std::deque me donne donc la possibilité d'insérer rapidement de nouveaux items aux extrémités, mais aussi de parcourir la liste au complet grâce à GetAll() et de supprimer un item spécifique qui n'est pas simplement le premier dans la liste si c'est celui que le joueur veut utiliser.

1.4 Structure d'Items (struct Item)

- Un enum class ItemType pour donner à un item une des 2 fonctionnalités possibles (Health, Buff)
 - Un int itemId pour pouvoir trouver l'item dans l'inventaire du joueur
 - Un float itemPower pour décider de combien l'item doit guérir ou buff le joueur
 - Un std::string pour la description de l'item aussi préformaté lors de la création pour faciliter l'affichage dans l'interface.
-

2. Choix techniques et Architecture

2.1 Intégration de SFML 3.0

Pour le rendu graphique, j'ai choisi la librairie **SFML** car elle permet une gestion simple du fenêtrage et des primitives 2D (Rectangles pour le joueur et les ennemis). L'architecture du moteur sépare clairement la logique (Update) du rendu (Render).

- **Rendu** : À chaque itération de la boucle, l'écran est effacé (clear), les entités sont mises à jour, puis dessinées (draw), et enfin la fenêtre est rafraîchie (display).
- **Compatibilité** : Le code a été adapté pour être compatible avec la nouvelle syntaxe de gestion d'événements de SFML 3 (event->is<...> et event->getIf<...>).

2.2 Boucle de Jeu et DeltaTime (dt)

Une exigence critique était que les actions "ne doivent pas être instantanées". Pour respecter cela, j'ai implémenté une gestion du temps basée sur une horloge (sf::Clock).

- Le déplacement ne se fait pas par téléportation, mais par une interpolation vectorielle calculée à chaque frame : $\text{\$Position} += \text{Direction} * \text{Vitesse} * \text{dt}$.
- Cela garantit que le jeu tourne à la même vitesse, peu importe la puissance de l'ordinateur (Frame-rate independent).

2.3 Machine à État Simple (isBusy)

Pour assurer l'exécution séquentielle des actions (une à la fois), j'ai mis en place un indicateur d'état isBusy dans la boucle principale.

- **Input** : Le joueur peut entrer des commandes à tout moment; elles sont immédiatement ajoutées à la deque.
- **Execution** : Le moteur ne dépile ("pop") la prochaine action que si isBusy est false. Dès qu'une action commence, isBusy devient true jusqu'à ce que la condition de fin (distance < 5px ou timer <= 0) soit atteinte.

2.4 Parsing de Commandes (std::stringstream)

Pour interpréter les entrées textuelles du joueur (ex: "walk 500 300"), j'ai utilisé std::stringstream. Cela permet de découper facilement la chaîne de caractères pour isoler la commande (le verbe) et ses arguments (coordonnées ou ID), tout en validant que les types de données sont corrects avant de créer l'action.

3. Problèmes rencontrés

3.1 Gestion des DLLs et Git

Un problème majeur rencontré a été la gestion des dépendances SFML lors du travail d'équipe. Initialement, les fichiers .dll nécessaires à l'exécution (sfml-graphics-d-3.dll, etc.) étaient ignorés par le fichier .gitignore, qui exclut le dossier de build x64/Debug.

- **Conséquence** : Le projet fonctionnait sur ma machine, mais plantait sur celles des coéquipiers.
- **Solution** : J'ai dû modifier manuellement le .gitignore pour forcer l'inclusion des fichiers binaires (*.dll) et de la police (*.ttf) afin d'assurer la portabilité du projet sur GitHub.

3.2 Gestion des Items dans l'inventaire

J'ai eu plusieurs problèmes avec l'inventaire, que ce soit pour display et bien mettre à jour l'inventaire quand un item est pris ou utilisé, ou pour qu'un joueur ne puisse pas utiliser le même item 2 fois

- **Conséquences** : L'inventaire est un gros feature quand même important, et sans lui il manque beaucoup, et avec tous les bogues de display qui montrent des items plus présents dans l'inventaire ou permetts à un joueur d'utiliser plusieurs fois le même item, ceci est un problème
- **Solution** : Pour le premier problème, j'ai regardé le code de la queue pour voir comment la liste était mise à jour quand une action était ajoutée et

supprimée et je me suis inspiré de ceci pour que les actions get et use refresh le display de la liste. Pour le deuxième problème, je me suis rendu compte que je n'avais simplement rien qui se passait si mon get item by id ne marchait pas. Quand je faisais le get item by id pour supprimer l'item quand il était utilisé, si un mauvais id était entré, soit le programme crashait, ou soit un item venait d'être utilisé et donc le cache de l'item utilisé était réutilisé. J'ai donc ajouté le fait que le get item by id retourne un item void avec le id -1 si non trouvé, et je vérifie dans le use que ce n'est pas cet item qui se retrouve à être appelé pour être utilisé.

4. Répartition des tâches

[Mohaned] : Architecture Moteur & Rendu

- Mise en place de l'environnement Visual Studio, SFML et Git.
- Implémentation de la structure de données ActionQueue (std::deque).
- Création de la boucle de jeu ("Game Loop") et gestion du temps (DeltaTime).
- Développement du système de mouvement vectoriel
- Système d'analyse syntaxique (Parsing) des commandes textuelles.
- Affichage graphique (HUD, Joueur, File d'attente).

[Loïc] : Ennemis, npc, combat, dialogue et game over

- Grandes modifications à la classe Character pour qu'elle puisse facilement être utilisée pour créer le joueur, les ennemis et le npc. Elle inclut notamment directement un rectangle et deux textes ainsi que des fonctions pour facilement instancier ceux-ci, permettant de créer et personnaliser un nouveau joueur/ennemi/npc ainsi que leur dialogue/barre de PV en appelant quelques fonctions seulement.
- Ajout de l'action "attack", qui démarre un combat avec un ennemi si on est assez proche de lui. Le joueur et l'ennemi commencent alors à se donner des coups en alternance, chaque coup prenant 1 seconde à compléter. Le combat se termine lorsque le joueur ou l'ennemi n'a plus de pv.

- Affichage simple des pv du joueur et des ennemis avec un texte intégrer à la classe Character.
- Gestion de la mort des ennemis et du joueur. Les ennemis ne sont plus draw et on ne peut plus les attaquer ou leur parler. Pour la mort du joueur, un texte GAME OVER s'affiche avec un décompte de 5 secondes, après lequel le jeu se ferme.
- Ajout de l'action "talk". Si le joueur est assez proche, fais afficher au-dessus des ennemis et du npc un texte pendant quelques secondes.
- Création d'un npc. C'est simplement un Character comme les ennemis, mais sans la possibilité de l'attaquer.

[Frédéric] : Items, Inventaires, get et use, items à la mort d'un ennemi

- Création d'un système d'inventaire fonctionnant comme la queue d'actions, dont on peut rajouter des objets de type items, tout comme un inventaire de jeu vidéo
- Création d'items qui peuvent être utilisés soit pour guérir le joueur ou augmenter son dommage contre les ennemis.
- Ajout à la scène de 2 objets ajoutés dans un endroit aléatoire à chaque lancement du jeu, un de chaque type
- Ajout d'un système donnant au joueur un item aléatoire à la mort d'un ennemi
- Ajout de l'action get, qui permet au joueur de collecter un item s'il est assez proche lors de l'exécution de la commande
- Ajout de l'action use, qui, avec un id d'item, permet au joueur d'utiliser ses items présents dans son inventaire, chacun seulement une fois
- Fix de bogues et code général, entre autres pour la classe Character